create the `ClaimsPrincipal`. The difference is in how an API handles authorization failures.

When an API app generates a challenge response, it returns a `401 Unauthorized` error response to the caller. Similarly, when the app generates a forbid response, it returns a `403 Forbidden` response. The traditional web app essentially handled these errors by automatically redirecting unauthorized users to the login or "access denied" page, but the API app doesn't do this. It's up to the client-side SPA or mobile app to detect these errors and handle them as appropriate.

TIP This difference in authorization behavior is one of the reasons I generally recommend creating separate apps for your APIs and Razor pages apps; it's possible to have both in the same app, but the configuration is often more complex.

The different behavior between traditional web apps and SPAs can be confusing initially, but you generally don't need to worry about that too much in practice. Whether you're building an API app or a traditional MVC web app, the authorization code in your app looks the same in both cases. Apply `[Authorize]` attributes to your endpoints, and let the framework take care of the differences for you.

NOTE In chapter 23 you saw how to configure ASP.NET Core Identity in a Razor Pages app. This chapter assumes that you're building a Razor Pages app too, but the chapter is equally applicable if you're building an API, as you'll see in chapter 25. Authorization policies are applied in the same way, whichever style of app you're building. Only the final response of unauthorized requests differs.